Tab 1

Smart School ERP — Project Status

Last updated: May 2, 2026 **Stack:** Express 5 + React + Vite + PostgreSQL + Drizzle ORM (pnpm monorepo)

✅ সম্পন্ন কাজ (Completed)

1. Authentication System

- JWT-based login (`POST /api/auth/login`)
- `/api/auth/me` endpoint to fetch current user
- Password hashing with HMAC-SHA256 (no external library)
- Token stored in `localStorage` as `erp_token`
- Auto-attach Bearer token on every API call via `setAuthTokenGetter`
- Login page with default credential hints

2. Role-Based Access Control (RBAC)

- 5 roles defined: `SUPER_ADMIN`, `TEACHER`, `ACCOUNTANT`, `PARENT`, `STUDENT`
- `requireAuth` middleware — verifies JWT on all protected routes
- `requireAdmin` middleware — SUPER_ADMIN only (users, audit, etc.)
- `requireFinance` middleware — SUPER_ADMIN + ACCOUNTANT (finance routes)
- Teacher class-locking: teachers only see their own assigned class
- Frontend route guards via `canAccessRoute()` → `AccessDenied` page for blocked routes
- Role color badges in sidebar (purple = SUPER_ADMIN, blue = TEACHER, green = ACCOUNTANT)

Route	SUPER_ADMIN	TEACHER	ACCOUNTANT

/dashboard	✅	✅	✅
/students	✅	own class only	❌
/attendance	✅	own class only	❌
/classes	✅	view only	❌
/finance	✅	❌	✅
/users	✅	❌	❌
/audit	✅	❌	❌

3. Student Management

- CRUD: create, list, update, delete students
- Auto-generated student ID (`STU-2026-XXXX`)
- Search by name or student ID
- Filter by class and enrollment status
- Admission date tracking
- Parent/guardian contact details

4. Class Management

- Create and update classes with grade level and section
- Assign teacher to class
- Live student count per class
- Teacher restricted to view-only (no create/update for TEACHER role)

5. Attendance System

- Mark individual attendance (PRESENT, ABSENT, LATE, EXCUSED)
- **Bulk attendance** for full class in one API call
- Filter by class, date, student
- Teacher can only mark attendance for their own class
- Attendance summary widget on dashboard (per class, rate %)

6. Finance Module

- **Fee Types** — configurable fee categories with amounts (recurring/one-time)
- **Invoices** — auto-generated invoice numbers (`INV-202605-XXXX`), linked to student + fee type
- **Transactions** — record payments, auto-update invoice paid amount + status
- Invoice statuses: PENDING → PAID / OVERDUE
- Finance stats on dashboard (monthly revenue, pending invoices, overdue)
- Revenue trend chart (6-month line chart, collected vs pending)

7. Staff/User Management

- CRUD for staff accounts (SUPER_ADMIN only)
- Role assignment during creation
- Active/inactive status toggle
- Conflict check on email uniqueness

8. Dashboard

- Stats cards: total students, teachers, classes, attendance rate, monthly revenue, total revenue, pending/overdue invoices, new admissions
- Revenue trend chart (Recharts)
- Today's attendance breakdown table (per class)
- **Live Audit Feed** (SUPER_ADMIN only) — last 5 system events, auto-refreshes every 30s
- Recent Activity card (TEACHER / ACCOUNTANT view)

9. Audit Log System

- `audit_logs` table in PostgreSQL
- `audit()` helper — fire-and-forget, never blocks requests
- Auto-logging in all write routes:
  - Auth: `LOGIN`, `LOGIN_FAILED`
  - Users: `CREATE`, `UPDATE`, `DELETE`
  - Students: `CREATE`, `UPDATE`, `DELETE`
  - Classes: `CREATE`, `UPDATE`
  - Attendance: `BULK_ATTENDANCE`
  - Finance: `CREATE` (fee type, invoice), `PAYMENT` (transaction)
- `GET /api/audit-logs` — filterable by action, entity, date range; paginated
- **Audit Log page** (`/audit`) — SUPER_ADMIN only
  - Filter bar: action type, entity, date from/to
  - Color-coded action badges (CREATE=green, UPDATE=yellow, DELETE=red, PAYMENT=purple)

    - **Export to CSV** — respects active filters, downloads all matching records
    - Pagination (30 per page)
- Live Audit Feed widget on Dashboard

10. Backend Architecture

- Express 5 + esbuild (single `dist/index.mjs`)
- Routes organized into subdirectories:
- ```
  routes/
  ```
- ```
  auth/index.ts        health/index.ts
  ```
- ```
  users/index.ts       dashboard/index.ts
  ```
- ```
  classes/index.ts     audit/index.ts
  ```
- ```
  students/index.ts    finance/index.ts
  ```
- ```
  attendance/index.ts
  ```
- OpenAPI 3.1 spec → Orval codegen → React Query hooks + Zod schemas
- Drizzle ORM for all DB queries (no raw SQL)
- Pino structured logging on all requests

11. Frontend Architecture

- React + Vite + TypeScript
- Shadcn UI components (Card, Button, Input, Select, Dialog, Toast, Badge, Skeleton)
- Wouter for routing (lightweight, path-based)
- React Query for all server state
- Responsive sidebar layout with mobile hamburger menu
- Breadcrumb in header

❌ বাকি কাজ (Remaining / Planned)

High Priority

- **Parent/Student portal** — separate login view for parents to see their child's attendance and invoices
- **Password change** — authenticated users should be able to change their own password
- **Forgot password / reset flow** — email-based reset (needs SMTP integration)

- **Overdue invoice auto-marking** — a cron job that marks PENDING invoices as OVERDUE past due date

Medium Priority

- **Notifications** — in-app notification bell (new invoice, overdue, etc.)
- **Student document uploads** — profile photo, admit card, birth certificate
- **Timetable / Schedule** — class schedule per grade
- **Subject & Marks** — exam results, grade cards
- **Bulk student import** — CSV upload to add many students at once
- **SMS/WhatsApp alerts** — notify parents on absence (Twilio / local SMS gateway)

Low Priority

- **Multi-school / Multi-branch** support (tenant isolation)
- **Dark/light theme toggle** for users
- **Attendance QR code** — generate QR for each student for faster scanning
- **Annual calendar** — holidays, events, exam schedule
- **Fee discount / scholarship** — partial discount on invoices

---

🔐 Default Credentials

Role	Email	Password
Super Admin	admin@school.edu	3
Teacher	sarah.ahmed@school.edu	teacher123
Accountant	accountant@school.edu	accountant123

🗄️ Database Tables

Table	Description
`users`	Staff accounts (all roles)
`classes`	Grade-level classes with teacher assignment
`students`	Student profiles + enrollment
`attendance`	Daily per-student attendance records
`fee_types`	Configurable fee categories
`invoices`	Student fee invoices with status tracking
`transactions`	Payment records linked to invoices
`audit_logs`	Immutable event trail for all system actions

📁 Key Files

```
artifacts/

 api-server/src/

   routes/              ← all route subdirectories

   middlewares/         ← requireAuth, requireRole

   lib/

     auth.ts            ← JWT create/verify, password hash

     audit.ts           ← audit log helper
```

```
school-erp/src/
  pages/              ← one file per page
  components/
    Layout.tsx        ← sidebar + header
    AccessDenied.tsx  ← shown for blocked routes
  lib/
    auth.tsx          ← AuthContext, ROLE_CONFIG, usePermissions
lib/
db/src/schema/        ← Drizzle table definitions
api-spec/             ← OpenAPI 3.1 YAML (source of truth)
api-zod/              ← generated Zod schemas
api-client-react/     ← generated React Query hooks
```

⚡ Commands

```
# Push DB schema changes
pnpm --filter @workspace/db run push

# Regenerate API hooks from OpenAPI spec
pnpm --filter @workspace/api-spec run codegen

# Full typecheck
pnpm run typecheck

# Individual workflow restarts (done via Replit UI)
```

```
# API server: artifacts/api-server: API Server

# Frontend:   artifacts/school-erp: web
```

🔗 Routing

```
/            → → /dashboard (if logged in) or /login

/login       → LoginPage

/dashboard   → DashboardPage      (all roles)

/students    → StudentsPage       (SUPER_ADMIN, TEACHER)

/attendance  → AttendancePage     (SUPER_ADMIN, TEACHER)

/classes     → ClassesPage        (SUPER_ADMIN, TEACHER)

/finance     → FinancePage        (SUPER_ADMIN, ACCOUNTANT)

/users       → UsersPage          (SUPER_ADMIN)

/settings    → SettingsPage       (SUPER_ADMIN)

/audit       → AuditLogPage       (SUPER_ADMIN)
```

API routes all served under `/api` prefix (proxied to port 8080).

🚀 ADVANCED SYSTEM INSTRUCTION: ENTERPRISE B2B SAAS REFACTORING
Project Context: Mid-development Educational ERP (Django + Next.js). Completed phases: Core CRUD, Auth, Finance.
Current Goal: Refactor into a highly secure, Multi-Tenant SaaS platform with Dynamic Subdomain Routing, Offline-First Sync, and an IoT Asset Management module.
CRITICAL RULES FOR AI AGENT:

Zero Data Leakage: Ensure strict schema isolation. Tenant A must NEVER access Tenant B's data.
Do Not Break Existing Logic: You are extending the architecture, not rewriting business logic.
Execution: Execute ONE step at a time. Provide the code, explain the changes, and wait for my explicit confirmation before moving to the next step.
STEP 1: MULTI-TENANT ARCHITECTURE (django-tenants)
We will isolate data at the PostgreSQL schema level.
Install & Configure: Add django-tenants. Refactor settings.py to use TenantMainMiddleware.
App Splitting: Define SHARED_APPS (e.g., tenants, users, core) and TENANT_APPS (e.g., academics, students, finance).
Models: Create Client(TenantMixin) and Domain(DomainMixin). Include fields in Client for school_name, primary_color_hex, logo_url, and contact_email.
Migration Protocol: Provide the exact terminal commands to drop the existing DB, recreate it, and run migrate_schemas --shared.
STEP 2: EDGE ROUTING & DYNAMIC THEMES (Next.js)
The frontend must dynamically adapt based on the subdomain (e.g., dps.ourerp.com).
Next.js Middleware (src/middleware.ts): Write a middleware that detects the hostname/subdomain. Rewrite the URL internally so the app knows which tenant context to load (e.g., rewrite tenant1.app.com to /app/tenant1/).
Theme Engine: Create a layout wrapper that fetches the Client data from the backend using the detected subdomain. Inject primary_color_hex into the DOM as native CSS variables so Tailwind classes (e.g., bg-primary) automatically adapt to the school's brand.
STEP 3: OFFLINE-FIRST SYNC ENGINE (PWA + RxDB/Dexie)
Ensure the app functions during internet outages and syncs gracefully.
PWA Setup: Implement next-pwa with a custom service worker.
Local State (Dexie.js): Create a local database schema matching the critical Django models (Attendance, Invoices).
Mutation Queue: When offline, save API POST/PUT requests (like taking attendance) to IndexedDB.
Background Sync: Write a React hook (useSyncEngine) that listens for the window.addEventListener('online') event and pushes the queued mutations to the Django API automatically.
STEP 4: IOT & HARDWARE MAINTENANCE (WebRTC Ready)
Prepare the system for school hardware management.
Django App (apps/assets_iot): Create models for HardwareAsset (Type, IP Address, MAC Address, Location, Status) and MaintenanceLog.

Camera Streaming Scaffold: Create a Python utility or WebSockets/Django Channels routing scaffold capable of receiving an RTSP stream from an IP Camera and preparing it for WebRTC consumption on the Next.js frontend.
Frontend Dashboard: Create an "Asset Management" UI using Shadcn Data Tables to display hardware health, warranty expiries, and live status.
EXECUTION COMMAND:
Start with STEP 1. Analyze my current project structure. Give me the settings.py updates and the new models.py for the tenants app. Wait for my "Proceed" before generating migration commands.
💡

Future Ai Plan For school management

6. AI & AUTONOMOUS AGENTS ARCHITECTURE (THE 2030 EDGE)

Prepare the system architecture to support Local Open-Source LLMs (Llama-3/Mistral) and Autonomous Agents for predictive analytics and automated reporting.

- **AI Module Setup:** Create a dedicated Django app named `apps/ai_core/`.
- **Dependencies:** Add `transformers`, `langchain`, and `torch` to `requirements.txt`. Configure the environment to support future GPU acceleration (e.g., local RTX 8000 or cloud Vast.ai instances).
- **Autonomous Agent Simulation:** In `apps/ai_core/tasks.py`, define a Celery periodic task (`@shared_task`) named `agent_generate_daily_summary`. This agent should autonomously fetch today's total fee collections and attendance records from the database, format them into a natural language summary, and log or send it to the Super Admin dashboard.
- **Predictive Endpoints:** Create a placeholder DRF API endpoint `GET /api/ai/predict-dropout/` that will eventually use a fine-tuned model to analyze student attendance/marks and predict the risk of dropping out.

Tab 2

ADVANCED SYSTEM INSTRUCTION: ENTERPRISE B2B SAAS REFACTORING

Project Context: Mid-development Educational ERP (Django + Next.js). Completed phases: Core CRUD, Auth, Finance.

Current Goal: Refactor into a highly secure, Multi-Tenant SaaS platform with Dynamic Subdomain Routing, Offline-First Sync, and an IoT Asset Management module.

CRITICAL RULES FOR AI AGENT:

1. **Zero Data Leakage:** Ensure strict schema isolation. Tenant A must NEVER access Tenant B's data.
2. **Do Not Break Existing Logic:** You are extending the architecture, not rewriting business logic.
3. **Execution:** Execute ONE step at a time. Provide the code, explain the changes, and wait for my explicit confirmation before moving to the next step.

STEP 1: MULTI-TENANT ARCHITECTURE (`django-tenants`)

We will isolate data at the PostgreSQL schema level.

- **Install & Configure:** Add `django-tenants`. Refactor `settings.py` to use `TenantMainMiddleware`.
- **App Splitting:** Define `SHARED_APPS` (e.g., `tenants`, `users`, `core`) and `TENANT_APPS` (e.g., `academics`, `students`, `finance`).
- **Models:** Create `Client(TenantMixin)` and `Domain(DomainMixin)`. Include fields in `Client` for `school_name`, `primary_color_hex`, `logo_url`, and `contact_email`.
- **Migration Protocol:** Provide the exact terminal commands to drop the existing DB, recreate it, and run `migrate_schemas --shared`.

STEP 2: EDGE ROUTING & DYNAMIC THEMES (Next.js)

The frontend must dynamically adapt based on the subdomain (e.g., `dps.ourerp.com`).

- **Next.js Middleware (`src/middleware.ts`):** Write a middleware that detects the hostname/subdomain. Rewrite the URL internally so the app knows which tenant context to load (e.g., rewrite `tenant1.app.com` to `/app/tenant1/`).
- **Theme Engine:** Create a layout wrapper that fetches the `Client` data from the backend using the detected subdomain. Inject `primary_color_hex` into the DOM as native CSS

variables so Tailwind classes (e.g., `bg-primary`) automatically adapt to the school's brand.

STEP 3: OFFLINE-FIRST SYNC ENGINE (PWA + RxDB/Dexie)

Ensure the app functions during internet outages and syncs gracefully.

- **PWA Setup:** Implement `next-pwa` with a custom service worker.
- **Local State (`Dexie.js`):** Create a local database schema matching the critical Django models (Attendance, Invoices).
- **Mutation Queue:** When offline, save API POST/PUT requests (like taking attendance) to IndexedDB.
- **Background Sync:** Write a React hook (`useSyncEngine`) that listens for the `window.addEventListener('online')` event and pushes the queued mutations to the Django API automatically.

STEP 4: IOT & HARDWARE MAINTENANCE (WebRTC Ready)

Prepare the system for school hardware management.

- **Django App (`apps/assets_iot`):** Create models for `HardwareAsset` (Type, IP Address, MAC Address, Location, Status) and `MaintenanceLog`.
- **Camera Streaming Scaffold:** Create a Python utility or WebSockets/Django Channels routing scaffold capable of receiving an RTSP stream from an IP Camera and preparing it for WebRTC consumption on the Next.js frontend.
- **Frontend Dashboard:** Create an "Asset Management" UI using Shadcn Data Tables to display hardware health, warranty expiries, and live status.

EXECUTION COMMAND:

Start with STEP 1. Analyze my current project structure. Give me the `settings.py` updates and the new `models.py` for the tenants app. Wait for my "Proceed" before generating migration commands.
